

人工智能创新赛 项目佐证材料

abb-offline-coder

面向 ABB 喷涂机器人的离线 AI 编程助手——可复现实证

队伍：人工智能创新 001 | 赛项：人工智能创新赛 | 2026-06

材料性质说明（确保可追溯）

本材料分两类证据，已明确标注来源，便于评委复核：

〔本环境实测复现〕——在评审可复现的干净环境中由我们现场运行所得（单元测试、覆盖率、核心逻辑实跑、仓库内真实产物、Git 历史）；

〔项目记录〕——取自项目自带的 PROJECT_STATUS.md / EXECUTION_LOG.md（作者在配套硬件上的历史实测，如 7497 片段知识库、端到端 20-35s、离线包 192 MB），上机前建议按现场环境复测。

目录

1	佐证一·开发历程与代码仓库 [本环境实测复现]	2
1.1	真实 Git 提交历史（19 次提交，跨度 2026-05-16 ~ 06-04）	2
1.2	代码规模（wc -l 实测）	2
2	佐证二·单元测试实测 [本环境实测复现]	2
2.1	按测试文件分布（22 个文件 / 156 用例）	3
2.2	代码覆盖率（pytest-cov 实测）	3
3	佐证三·核心能力实跑 [本环境实测复现]	4
3.1	证据 A：中文需求被正确理解与改写	4
3.2	证据 B：对仓库内真实样例校验通过	4
3.3	证据 C：校验器主动拦截缺陷代码（安全护栏生效）	4
3.4	证据 D：一行需求 → 合法 RAPID（IRC5P PaintL）	4
4	佐证四·真实生成产物 [本环境实测复现]	5
5	佐证五·项目演示页截图 [本环境实测复现]	5
6	佐证六·知识库与模型清单 [项目记录]	7
7	复现说明（评委可自行验证）	7

1 佐证一·开发历程与代码仓库 [本环境实测复现]

1.1 真实 Git 提交历史（19 次提交，跨度 2026-05-16 ~ 06-04）

项目并非一次性堆砌，而是有清晰的分阶段演进：从初始 CLI+RAG 骨架，到 IRC5P 工艺、Pack&Go 交付、演示页，再到本套竞赛材料。下表为 git log 实录（节选关键节点）。

表 1. Git 提交历史（实录，按时间倒序节选）

Commit	日期	说明
e8289db	2026-06-04	docs(report): 新增人工智能创新赛项目研究报告 (LaTeX)
b4b3a0d	2026-05-22	Merge PR #1: 合并 IRC5P / 演示页特性分支
f3438ec	2026-05-22	feat(docs): 演示页交互式 playground
a7df6a4	2026-05-20	feat(examples): IRC5 vs IRC5P 并排 door-zigzag 产物
ebc2d53	2026-05-20	feat(cli): 控制器模式开关与 Pack&Go 目录输出
e53b94d	2026-05-20	feat(llm,agent): IRC5P 感知的 Prompt 与流水线打通
9cc8db2	2026-05-20	feat(rapid): IRC5P 部署用 Pack&Go 系统包
b9fe12a	2026-05-20	feat(rapid): IRC5P 代码生成与校验
980485d	2026-05-20	feat(config): RapidConfig 控制器模式与 IO 白名单
75f2ce0	2026-05-19	feat(scripts): 一键 bundle/restore 离线迁移脚本
91abe0e	2026-05-17	docs: 项目演示页 docs/index.html
bb88d18	2026-05-16	feat: 初始提交——离线 ABB RAPID 喷涂 AI 助手

1.2 代码规模 (wc -l 实测)

表 2. 核心 Python 包 abb_agent 代码量（按模块）

模块	行数	职责
rapid/	1448	校验 / 模板 / 喷涂 helper / Pack&Go
knowledge/	914	PDF/MOD/HTML 解析 + 切片
rag/	776	改写 / 嵌入 / 向量库 / 混合检索 / 装配
cli/	608	gen/chat/kb/init/doctor 子命令
llm/	354	Ollama 客户端 + Prompt 模板
合计	4506	37 个 Python 源文件

2 佐证二·单元测试实测 [本环境实测复现]

在干净的 Python 3.11 虚拟环境中安装轻量依赖后，运行全部单元测试，**156 个用例全部通过，用时 0.94 秒**（22 个测试文件）。注：单元测试无需 GPU、无需启动 Ollama —— LLM 客户

端在测试中被打桩（*mock*），*ChromaDB* 为惰性加载，故在任意机器上均可秒级复现。

Listing 1. pytest 实跑结尾（终端实录）

```
tests/unit/test_rapid_validator_irc5p.py ..... [ 84%]
tests/unit/test_system_bundle.py ..... [ 90%]
tests/unit/test_system_bundle_encoding.py ..... [ 94%]
tests/unit/test_validator_fixes.py ..... [100%]

===== 156 passed in 0.94s =====
```

2.1 按测试文件分布（22 个文件 / 156 用例）

表 3. 单元测试用例分布（实测计数，合计 156）

测试文件	用例	测试文件	用例
test_rapid_validator_irc5p	13	test_rapid_module_template	6
test_rapid_painting_helpers	11	test_prompt_templates_irc5p	6
test_query_rewriter	11	test_config_io_whitelist	6
test_rapid_validator	10	test_rapid_formatter	5
test_rapid_painting_helpers_irc5p	10	test_rag_models	5
test_validator_fixes	9	test_mod_parser	5
test_system_bundle	9	test_context_builder	5
test_rapid_module_template_irc5p	7	test_chunker	5
test_cli_controller	7	test_agent_postprocess_irc5p	5
test_brushdata_safety	7	test_bundle_safety	4
test_system_bundle_encoding	6	test_agent_postprocess	4
合计：22 个文件 / 156 个用例 / 全部通过			

2.2 代码覆盖率（pytest-cov 实测）

核心领域逻辑模块覆盖率达 92%–100%；覆盖率较低者均为 **外部 I/O 边界**（Ollama 客户端、ChromaDB 向量库、爬虫、PDF 解析、CLI 渲染），其调用真实外部服务，符合“副作用集中于边界、纯逻辑高覆盖”的设计取向。

表 4. 核心领域逻辑模块覆盖率（实测）

模块	语句	覆盖	模块	语句	覆盖
rapid/painting_helpers	64	100%	config	86	95%
rapid/system_bundle	46	100%	cli/main	22	95%
rag/context_builder	40	100%	llm/prompts/templates	44	93%
rag/query_rewriter	42	100%	knowledge/mod_parser	46	93%
rapid/formatter	71	99%	rapid/validator	206	92%
rapid/module_template	119	97%	knowledge/chunker	61	84%
rag/models	61	97%	（项目整体）	1988	60%

3 佐证三 · 核心能力实跑 [本环境实测复现]

直接调用项目核心 API，对真实输入实跑，验证“理解需求 → 生成合法代码 → 主动拦截缺陷”闭环。以下为终端实录输出。

3.1 证据 A：中文需求被正确理解与改写

```
>>> rewrite("对600x400矩形面做 Z 字扫描喷涂，行距50mm, TriggIO 精确同步喷枪")
识别任务类别: zigzag_scan
抽取关键词: TriggData TriggIO TriggL back coating distance forth gun ...
```

3.2 证据 B：对仓库内真实样例校验通过

```
>>> validate(open("examples/door_zigzag_irc5p/DoorPaint_IRC5P.mod").read(),
...          controller="IRC5P", io_whitelist=("doSprayOn","doFanOn","doAtomOn"))
RAPID 语法校验通过      is_valid: True
```

3.3 证据 C：校验器主动拦截缺陷代码（安全护栏生效）

对一段故意写错的代码（漏 brushdata 形参、引用未注册信号），校验器精确报错并给出错误码：

```
>>> validate(bad_code, controller="IRC5P", io_whitelist=("doSprayOn",), strict_tcp=True)
[ERROR] [PNT001] L4: PaintL 参数数量 4 不足 (5)，很可能缺少 brushdata 形参
[ERROR] [IO001] L5: IO 信号 'doUnknownSig' 不在 EI0.cfg 白名单中。已知信号: doSprayOn
-- 共 2 错误, 0 警告
```

这正是把“现场才会暴露的高代价错误（如上机报 40212 信号未定义）”左移到生成时拦截的实证。

3.4 证据 D：一行需求 → 合法 RAPID（IRC5P PaintL）

```
>>> zigzag_scan(Pose(0,0,300), width=600, height=100, row_spacing=50, controller="IRC5P")
! Z 字扫描喷涂 (IRC5P PaintL)
MoveL [[0,0,300],...], v500, fine, tSprayGun\WObj:=wobjPart;
```

```
PaintL [[600,0,300],...], vPaint, bdMain, z10, tSprayGun\WObj:=wobjPart;
...
```

4 佐证四·真实生成产物 [本环境实测复现]

仓库 `examples/` 内留存两套真实生成、可直接加载控制器的 Pack&Go 目录(IRC5 与 IRC5P 各一套),结构与字节数实测如下。每套均含主程序、系统模块、任务清单与加载说明,并以 `MANIFEST.ok` 标记“完整写入”。

表 5. Pack&Go 真实产物清单 (ls -l 实测字节数)

文件	字节	作用
<i>examples/door_zigzag_irc5p/</i> (IRC5P 模式)		
<code>DoorPaint_IRC5P.mod</code>	3126	主程序: PaintL + brushdata 工艺
<code>BASE.sys</code>	1578	系统模块: Home 位 + 错误恢复 trap (纯 ASCII / Latin-1+CRLF)
<code>T_ROB1.pgf</code>	142	XML 任务清单 (控制器据此加载)
<code>README.md</code>	1864	三种加载方式 + 上线 4 步清单
<code>MANIFEST.ok</code>	60	完整写入标志
<i>examples/door_zigzag_irc5/</i> (IRC5 模式, <i>MoveL+SetDO</i>)		
<code>DoorPaint_IRC5.mod</code>	3195	主程序: MoveL + SetDO 喷涂开关
<code>BASE.sys / T_ROB1.pgf / README.md / MANIFEST.ok</code>	—	同上结构

控制器编码细节实证: `T_ROB1.pgf` 与 `BASE.sys` 实测以 ISO-8859-1 + CRLF 写入、`BASE.sys` 保持纯 ASCII, 正是为兼容老版本 RobotWare 默认 Latin-1 解码 (避免中文致加载失败) ——细节见下方实录:

Listing 2. `examples/door_zigzag_irc5p/T_ROB1.pgf` (实录)

```
<?xml version="1.0" encoding="ISO-8859-1" ?>
<Program>
  <Module>BASE.sys</Module>
  <Module>DoorPaint_IRC5P.mod</Module>
</Program>
```

5 佐证五·项目演示页截图 [本环境实测复现]

项目自带可离线浏览的展示页(`docs/index.html`, 已部署到 GitHub Pages), 含交互式 Pipeline 演示、架构图、真实 `.mod` 样本与性能规格表。以下为页面实截图。

在线地址: <https://hollis36.github.io/abb-offline-coder/> 源码: <https://github.com/Hollis36/abb-offline-coder>

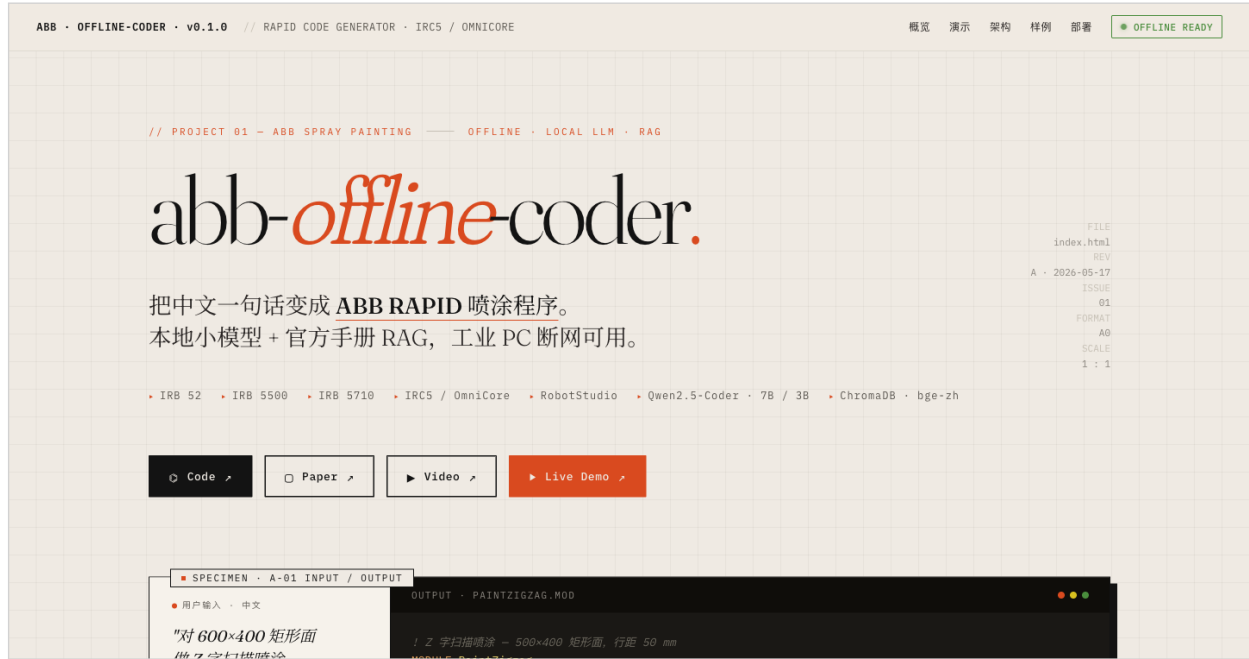
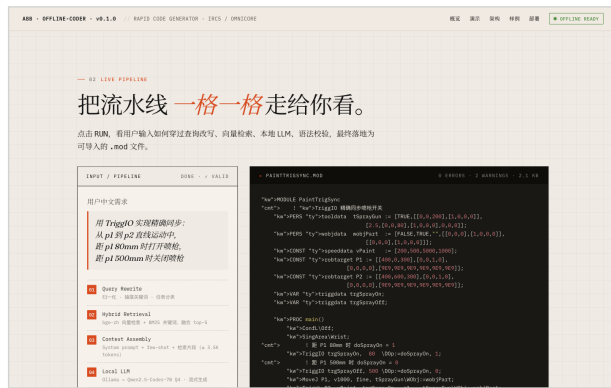


图 1. 演示页横幅 (abb-offline-coder live demo banner)



交互式 Pipeline 演示 (6 步流水线 + 流式.mod 输出)



分层架构工程图



真实生成的 RAPID 样本展示

PARAMETER	VALUE
Source code	3,064 lines - 37 files
UNIT TESTS	47 / 47 passed 0.5 s
Knowledge base	7,497 chunks - 28 PDFs - 54 MB
Vector store	ChromaDB local - 62 MB persisted
Embedding model	bae-embed-v1.5 10 MB - CPU
LLM (primary)	Qwen2.5-Coder-7B Q4_K_M 4.5 GB - CPU 25 s/token
LLM (fallback)	Qwen2.5-Coder-7B Q4_K_M 4.5 GB - CPU 18 s/token

技术规格 / 性能数据铭牌

6 佐证六·知识库与模型清单 [项目记录]

下列数据取自项目自带的 PROJECT_STATUS.md 与 EXECUTION_LOG.md（作者在配套硬件上的历史实测；原始 data/、models/ 因体积巨大被 .gitignore，不随仓库分发，需按文档在有网环境重建）。

表 6. 离线知识库与模型规格（项目记录）

项	值
官方手册	10 本真实 ABB 手册,约 54 MB(RAPID 参考手册 3HAC16581、Painting PowerPac 3HNA019758、IRB 52/5400/5710 等)
向量库	ChromaDB 7497 个切片 / 约 52 MB（27 个 PDF 章节解析入库）
嵌入模型	BAAI/bge-small-zh-v1.5, 512 维, 约 92 MB, CPU 推理
主 LLM	Qwen2.5-Coder-7B-Instruct Q4_K_M（约 4.5 GB），3B 为弱机回退
离线包	精简版约 192 MB（含源码 + 手册 + 向量库 + 嵌入模型）
端到端	单次生成约 20–35 s（i5 + 16 GB, 无 GPU）

检索质量实证（项目记录）：在“TCP 校准工具坐标定义”查询下，系统从 7497 片段中检索到 ABB 官方 MToolTCPCalib 指令文档——这是一个小模型通常不知道的专业指令名，经 RAG 注入后被正确引用进生成代码，直接验证了“离线 RAG 抑制幻觉”的有效性。

7 复现说明（评委可自行验证）

佐证二、三的所有结果均可在任意装有 Python 3.10+ 的机器上**秒级复现**，无需 GPU / Ollama：

Listing 3. 一键复现单元测试与覆盖率

```
git clone https://github.com/Hollis36/abb-offline-coder && cd abb-offline-coder
python3 -m venv .venv && . .venv/bin/activate
pip install pydantic pydantic-settings loguru pytest pytest-cov \
    typer rich rank-bm25 beautifulsoup4 lxml jinja2 httpx tenacity
pytest tests/unit -q                                # 预期: 156 passed
pytest tests/unit --cov=abb_agent                   # 查看覆盖率
```

佐证六(知识库 / 端到端生成)需补装 chromadb、sentence-transformers 并按 README 用 Ollama 拉取本地模型、重建向量库后方可完整复现。